Markov Chain Monte Carlo

Matthew Stephens and Heejung Shim

Learning goals

Metropolis-Hastings Algorithm:

- Understand key ideas for the algorithm
- Be able to derive and implement the algorithm
- (not examinable; challenging) Obtain intuitions on the theory to support why the algorithm works

Gibbs sampler:

- Understand it is a special case of the Metropolis-Hastings Algorithm
- Understand its potential limitation compared to the Metropolis-Hastings Algorithm

Markov chain Monte Carlo (MCMC)

MCMC is a very widely-used technique for simulating samples from the posterior distribution.

The basic idea is to simulate a Markov chain, whose stationary distribution is the posterior distribution. By simulating the Markov chain for long enough, one obtains samples that are "approximately" from the posterior distribution.

- Metropolis–Hastings Algorithm
- Gibbs Sampling

Example:
$N(\theta, 1) \rightarrow \theta'$

▷ Notation: →

- $q(\theta, \theta')$ → transition or proposal density
  - ↳ transition density: given current state θ we can use q to generate θ'.

- Target distribution → posterior distrib.

$$p(\theta|x) \propto p(x|\theta)\, p(\theta) = \pi$$

↳ target dist.

Metropolis–Hastings Algorithm

Let $q(\theta, \theta')$ denote a transition density, which is a way of generating new states θ' given the current state θ.

Let π denote the target distribution, up to a constant of proportionality (e.g. to simulate from the posterior, $\pi = p(\theta)p(x|\theta)$).

- Target distribution → posterior distribution (up to a constant is fine!)

Metropolis Hasting algorithm idea

- Goal → Generate samples from target distribution

posterior dist. → $P(\theta|x) \propto p(\theta)\,p(x|\theta) = \pi(\theta)$ ↪ target d.

- Algorithm →

→ $\theta^{(t)}$ current value

$t = t+1$ → Generate candidate $\theta' \sim q(\theta^{(t)}, \theta')$

current value ↓ propose value ↓

↪ proposal distribution

→ Use acceptance probability to decide wether $\theta^{(t+1)} = \theta'$ or $\theta^{(t+1)} = \theta^{(t)}$

Metropolis-Hasting algorithm

goal → Simulate samples from → $\theta \sim \underbrace{P(\theta|x)}_{\text{posterior d.}} = \pi(\theta)$ (target)

$t = 0 \rightarrow \theta^{(0)}$ initial value

$t = 1 \rightarrow ? \ \theta^{(1)} \rightarrow$ ① Propose new value

$\theta' \sim q(\theta^{(0)}, \theta')$

② Acceptance of θ':

$$A = \min\left\{1, \frac{\pi(\theta')}{\pi(\theta^{(0)})} \cdot \frac{q(\theta', \theta^{(0)})}{q(\theta^{(0)}, \theta')}\right\}$$

$t = m \rightarrow \theta^{(m)}$

What is the probability A ?!

② Acceptance of θ':

$$A = \min\left\{1, \frac{\pi(\theta')}{\pi(\theta^{(0)})} \cdot \frac{q(\theta', \theta^{(0)})}{q(\theta^{(0)}, \theta')}\right\}$$

θ' with prob A

$\theta^{(0)}$ with prob 1-A

$\frac{\pi(\theta')}{\pi(\theta^{(t)})}$ → compare proposed with current

$\frac{q(\theta', \theta^{(0)})}{q(\theta^{(0)}, \theta')}$ → Correct for proposal asymmetry

With prob. A:

$\theta^{(1)} = \theta'$

$u \sim U(0,1)$ ⇒

$u \leq A$ ⇒ accept θ'

$u > A$ ⇒ stay at $\theta^{(0)}$

→ $q(\theta^{(0)}, \theta') = 0.8$ → twice as large for the reverse move

$q(\theta', \theta^{(0)}) = 0.4$

→ $\frac{q(\theta', \theta^{(0)})}{q(\theta^{(0)}, \theta')} = 0.5$ →

→ $q(\theta^{(0)}, \theta') = 0.5 \cdot q(\theta', \theta^{(0)})$!

the factor 0.5 reduces the acceptance probability of the forward move

▶ $\pi(\theta')/\pi(\theta) = \frac{P(\theta')\,P(x|\theta')}{P(\theta)\,P(x|\theta)}$ → We don't have to worry about $P(x)$ here!

Metropolis–Hastings Algorithm

Now consider the following Markov chain:

1. Start with an initial value $\theta^{(0)}$; set $t = 0$.
2. Given the current value of $\theta^{(t)} = \theta$, generate a proposed new value θ' according to $q(\theta, \cdot)$.
3. Define the acceptance probability A by

$$A = \min\left(1, \frac{\pi(\theta')q(\theta', \theta)}{\pi(\theta)q(\theta, \theta')}\right).$$

target distribution

Acceptance probability.

proposal density

4. With probability A set $\theta^{(t+1)} = \theta'$; otherwise set $\theta^{(t+1)} = \theta$.
5. Increase t; return to step 2.

Note: q is referred to as the *proposal distribution*. The probability A is the *acceptance probability*.

Example:

- $\theta \sim P(\theta|x) \propto P(\theta) P(x|\theta) = \pi(\theta)$
- Proposal distribution: $q(\theta, \theta') = N(\theta, 1) \Rightarrow$

$\theta' \sim N(\theta, 1)$ candidate

$t=0 \quad \theta^{(0)}$

$t=1 \quad \theta^{(1)}$

$\vdots$

$t=m \quad \theta^{(m)}$

$\vdots$

$\theta^{(m)}$ approx $\sim p(\theta|x)$ for sufficient large m.

- $\theta \sim P(\theta|x) \propto \underbrace{P(\theta)}_{\text{prior}} \underbrace{P(x|\theta)}_{\text{likelihood}} = \pi(\theta)$ → known

- Proposal distribution: $q(\theta, \theta') \rightarrow \theta' \sim N(\theta, 1)$ candidate.

$t=0$ $\theta^{(0)} = 1$ current value

$t=1$ $\theta^{(1)} \rightarrow$ candidate $\theta' \sim N(\theta^{(0)} = 1, 1) \rightarrow \theta' = 0.5$

candidate $t=1$

$\theta^{(0)} = 1$ current value $t=0$

$t=m$ $\theta^{(m)}$

$t \rightarrow \infty$

- How do we choose?!

$$A = \min\left(1, \frac{\pi(\theta')\, q(\theta', \theta)}{\pi(\theta)\, q(\theta, \theta')}\right)$$

How do we choose?!

$$A = \min\left(1, \frac{\pi(\theta')\, q(\theta', \theta)}{\pi(\theta^{(0)})\, q(\theta^{(0)}, \theta')}\right)$$

θ': 0.5, 0.5, 1

$\theta^{(0)}$: 1, 1, 0.5

suppose we get

$$= \min(1, 0.9)$$

$$\theta' \sim N(\theta^{(0)} = 1, 1)$$

$\theta' = 0.5$ candidate $t=1$

$\theta^{(0)} = 1$ current candidate

$$\rightarrow \theta^{(1)} = \begin{cases} 0.5 & \text{with prob } 0.9 \\ 1 & \text{with prob } 0.1 \end{cases}$$

$\theta^{(1)} = 0.5$ if the candidate is accepted.

Why does Metropolis–Hastings algorithm work?

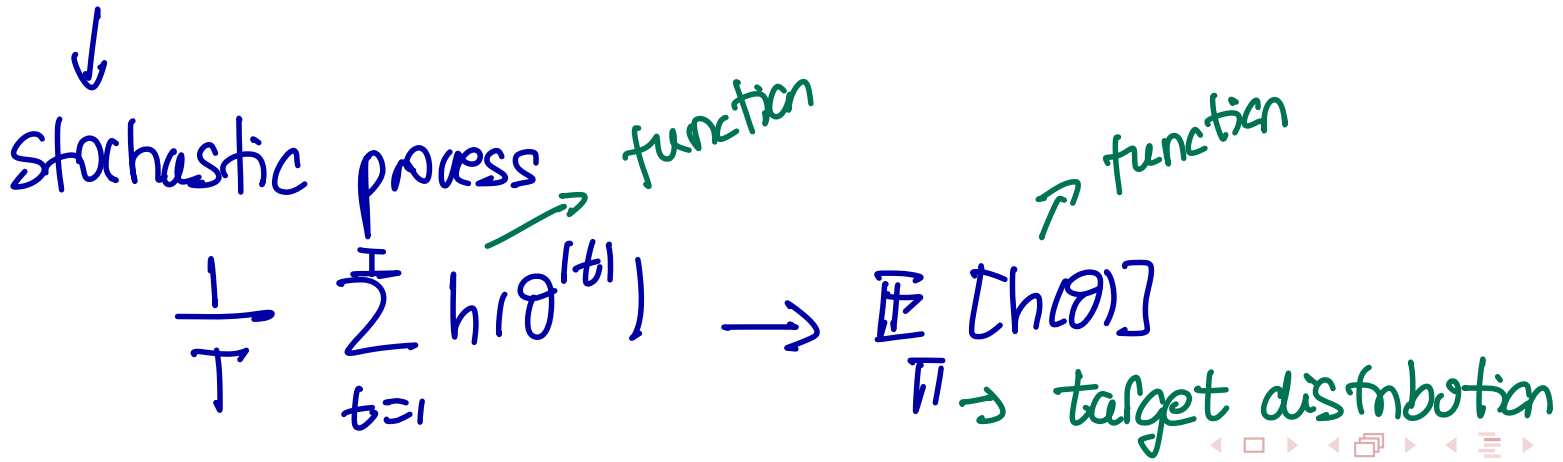
Note: See MarkovProcesses.pdf for definition and detailed explanation. The relevant theory for the convergence result is discussed at length in MAST30001.

- The process $\theta^{(t)}$ has stationary distribution π.
  - Proof: check detailed balance condition, $\pi(\theta)p(\theta \to \theta') = \pi(\theta')p(\theta' \to \theta)$.
- π will be limiting.
  - The process $\theta^{(t)}$ can stay where it is, it is aperiodic.
  - For all sensible choices of q it will also be irreducible (i.e., regardless the present state, it can reach any other state in finite time).
  - If a Markov process is aperiodic and irreducible then any stationary distribution is unique and limiting.
- The process is ergodic.
  - The process $\theta^{(t)}$ is aperiodic and irreducible, and π is a limiting distribution, then the process is ergodic.

Why does Metropolis–Hastings algorithm work?

Because the output process $\theta^{(t)}$ has a limiting distribution π, the output of the M-H algorithm will—in the limit—be a sequence of observations from the density π.

These observations are not independent, but because they come from an ergodic process we can still use them to estimate properties of π.



Metropolis–Hastings Algorithm

Remarks

1. For this algorithm to work q can be quite general, but we do need to know how to simulate from $q(\theta, \cdot)$ for every θ.
2. The algorithm only requires that we know π up to constant.

Choosing the proposal distribution q

Ideally you want q to have two properties

- It proposes large moves.
- It leads to high average acceptance probabilities, A.

In general these two properties are competing: it is difficult to achieve both. As a result, one generally tries to choose q so that it gives "intermediate" average values for A. [An exception to this is Gibbs sampling, where $A = 1$.]

Random Walk Metropolis-Hastings

Perhaps the most common type of MH sampler is the "random walk" MH sampler, where the proposal distribution q involves adding a symmetric random number (e.g. $U[-\epsilon, \epsilon]$ or $N(0, \epsilon^2)$) to the current value of θ.

In this case, the terms involving q in the acceptance probability cancel, due to symmetry. That is $q(\theta, \theta') = q(\theta', \theta)$ and

$$A = \min\left(1, \frac{\pi(\theta')}{\pi(\theta)}\right).$$

The value of ϵ determines the typical size of the proposed move, and hence the typical value for A.

Random Walk Metropolis-Hastings

The simpler form of A in the random-walk M-H makes it easier to see what the algorithm does.

The proposal process has no preferences regards which part of the parameter space it wanders in.

If the proposal process wants to move somewhere where π is larger, then $A = 1$ and the move is always accepted.

However if the proposal process wants to move somewhere where π is smaller, then $A < 1$ and there is a chance we stay where we are.

In this way the M-H process will spend relatively more time in regions where π is large.

Metropolis–Hastings Algorithm

Example: See `MHexample.pdf`.

Interactive visualization: `http://chi-feng.github.io/mcmc-demo/app.html?algorithm=RandomWalkMH&target=banana`

Let's go back to the Gibbs sampler

Suppose that $\theta = (\theta_1, \ldots, \theta_k)$ is a vector of unknown parameters. We aim to simulate samples from the posterior $p(\theta_1, \ldots, \theta_k | x)$. Then consider the following Markov chain

1. Start with an initial value $\theta^{(0)}$; set $t = 0$.
2. Sample $\theta_1^{(t+1)}$ from $p(\theta_1 | x, \theta_2^{(t)}, \ldots, \theta_k^{(t)})$.
3. Sample $\theta_2^{(t+1)}$ from $p(\theta_2 | x, \theta_1^{(t+1)}, \theta_3^{(t)}, \ldots, \theta_k^{(t)})$.
4. $\ldots$
5. Sample $\theta_k^{(t+1)}$ from $p(\theta_k | x, \theta_1^{(t+1)}, \ldots, \theta_{k-1}^{(t+1)})$.
6. Increase t and return to 2.

Note that at each step a component of the unknown parameter is sampled from its full conditional distribution, given the data, and the current value of all other components of the parameter.

The Gibbs sampler

Indeed, this Markov chain is a special case of MH sampling: using the full conditional distributions as the proposal distribution in an MH sampler gives acceptance probability $A = 1$.

Effect of correlation on the Gibbs sampler

The Gibbs sampler updates one co-ordinate at a time in order to obtain a new sample point.

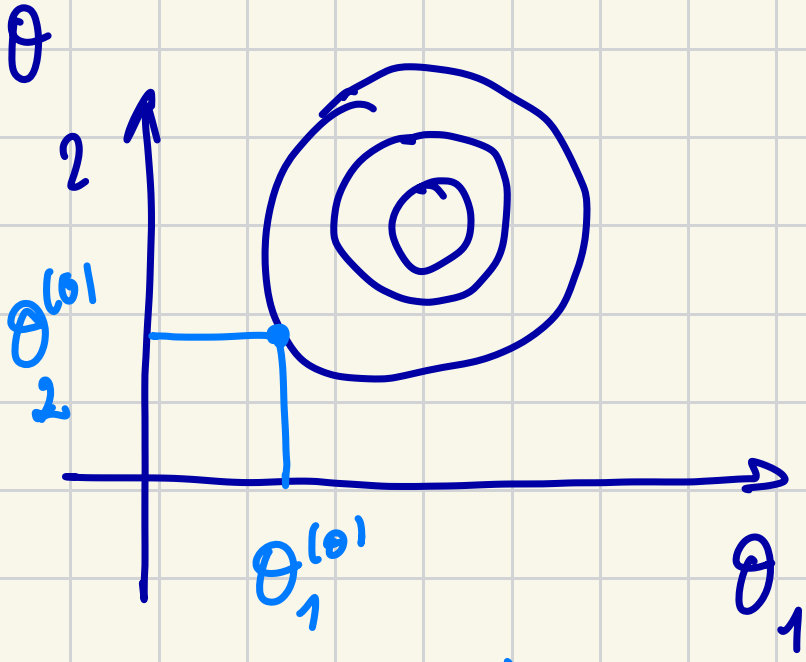
Strong correlation between the elements of $\theta = (\theta_1, \ldots, \theta_k)$ (that is, correlation between the co-ordinates of the target distribution π) will slow down the Gibbs sampler, so that it explores the sample space more slowly.

We illustrate this using the Gibbs sampler for a bivariate normal. See problems from the lab later.

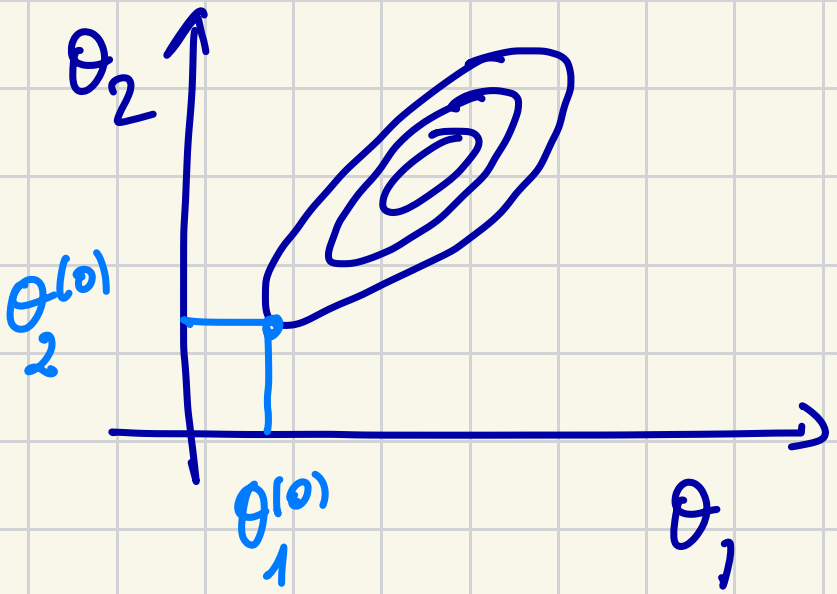
Understanding the effect of correlation:

$$\theta = (\theta_1, \theta_2) \sim P(\theta \mid x)$$

Case 1 (Weak correlation)



Case 2 (Strong correlation)



$$\theta^{(0)} = (\theta_1^{(0)}, \theta_2^{(0)})$$

$$\theta^{(1)} = (\theta_1^{(1)}, \theta_2^{(1)})$$

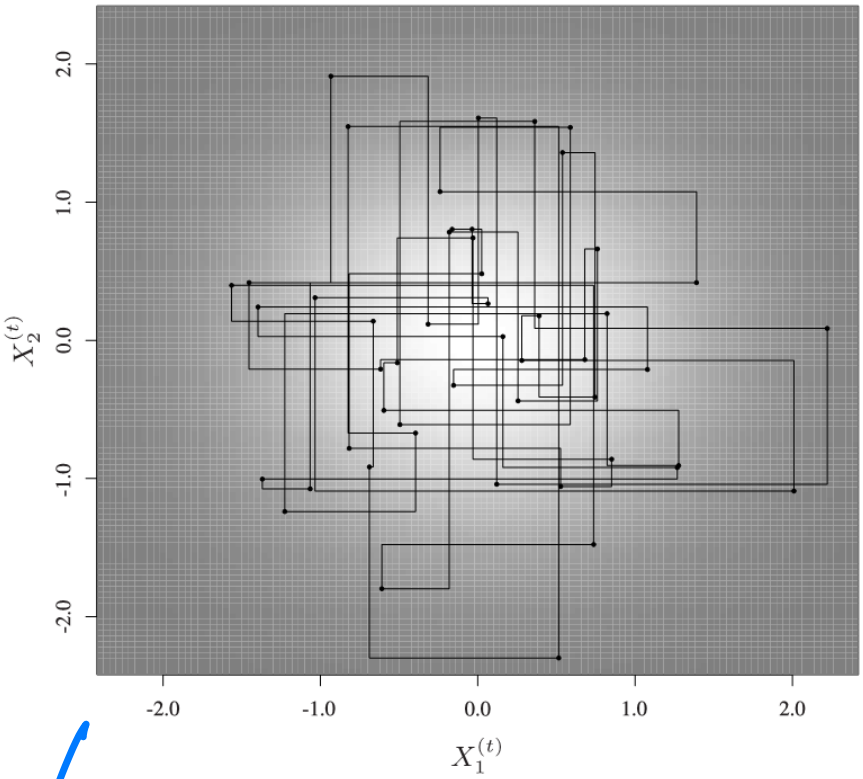
$$\theta_1^{(1)} \sim P(\theta_1 \mid x, \theta_2 = \theta_2^{(0)})$$

$$\theta_2^{(1)} \sim P(\theta_2 \mid x, \theta_1 = \theta_1^{(1)})$$

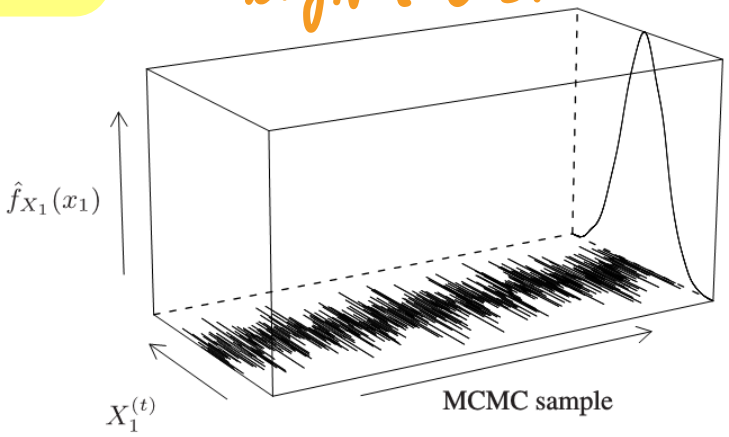
Effect of correlation on the Gibbs sampler

Figures from Ioana A. Cosma; Gibbs sampler aims to simulate (X_1, X_2) from bivariate standard normal distribution.

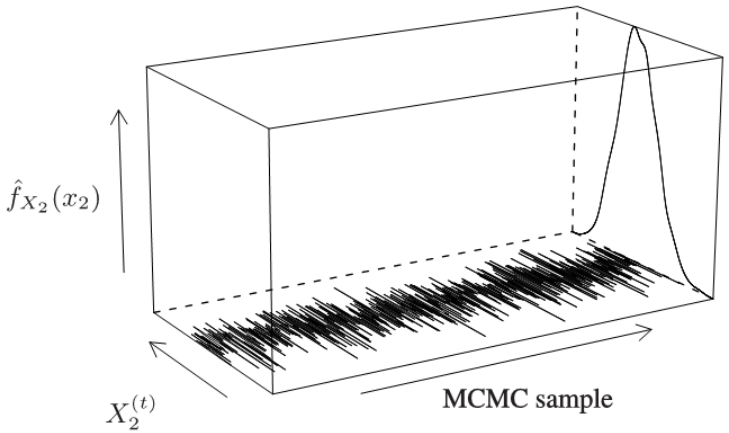
Marginal dist



(a) First 50 iterations of $(X_1^{(t)}, X_2^{(t)})$



(b) Path of $X_1^{(t)}$ and estimated density of X after 1,000 iterations



(c) Path of $X_2^{(t)}$ and estimated density of X_2 after 1,000 iterations

Does reasonable well around the target distrib.

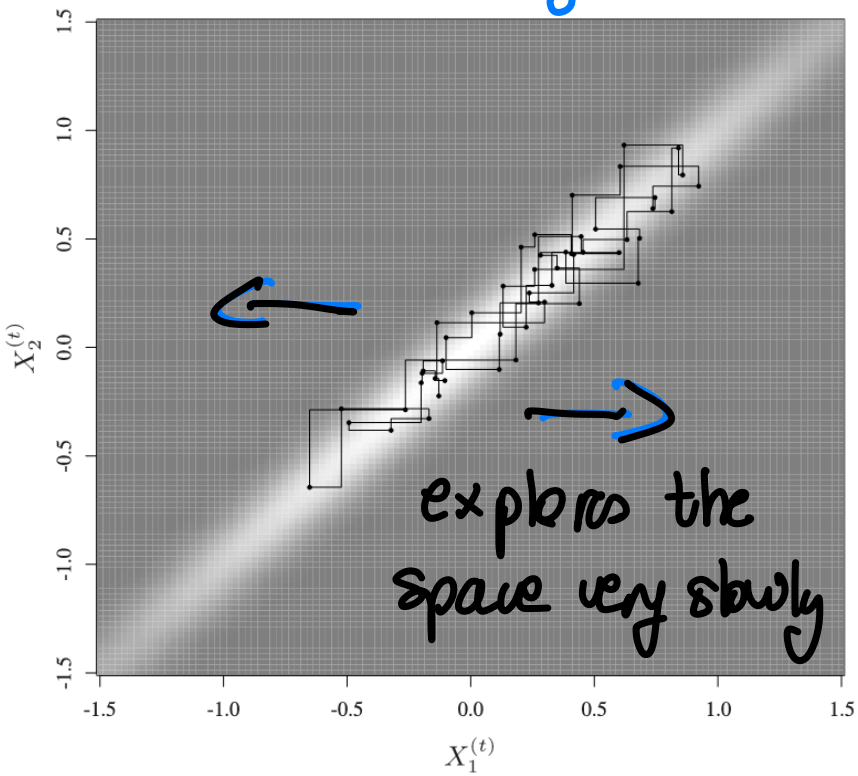
Correlation is not very strong → the sampler is moving through the distribution well.

Figure 4.4. Gibbs sampler for a bivariate standard normal distribution with correlation $\rho(X_1, X_2) = 0.3$.

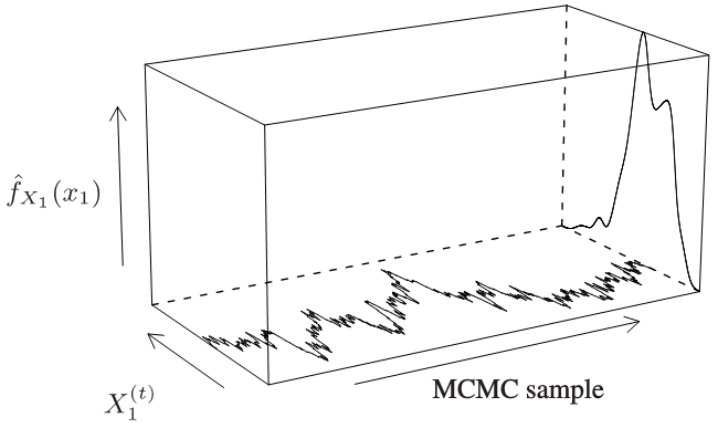
Effect of correlation on the Gibbs sampler

Figures from Ioana A. Cosma; Gibbs sampler aims to simulate (X_1, X_2) from bivariate standard normal distribution.

Target distribution is concentrated around a diagonal region

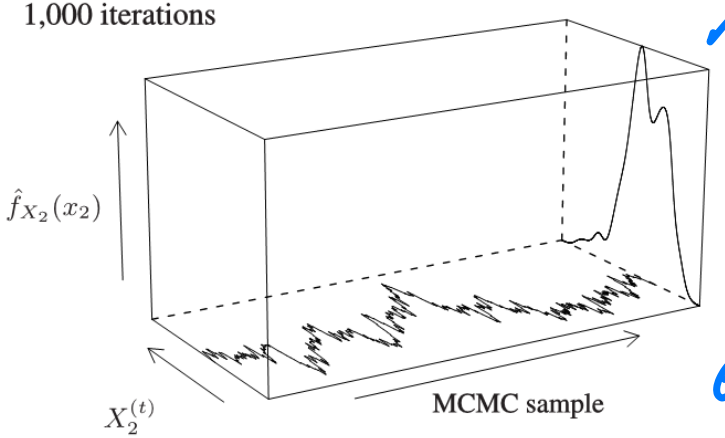


(a) First 50 iterations of $(X_1^{(t)}, X_2^{(t)})$



(b) Path of $X_1^{(t)}$ and estimated density of X_1 after 1,000 iterations

Successive MCMC samples are highly correlated



(c) Path of $X_2^{(t)}$ and estimated density of X_2 after 1,000 iterations

Slow exploration

Figure 4.5. Gibbs sampler for a bivariate normal distribution with correlation $\rho(X_1, X_2) = 0.99$.

The Gibbs sampler

Interactive visualization: `http://chi-feng.github.io/mcmc-demo/app.html?algorithm=GibbsSampling&target=banana`

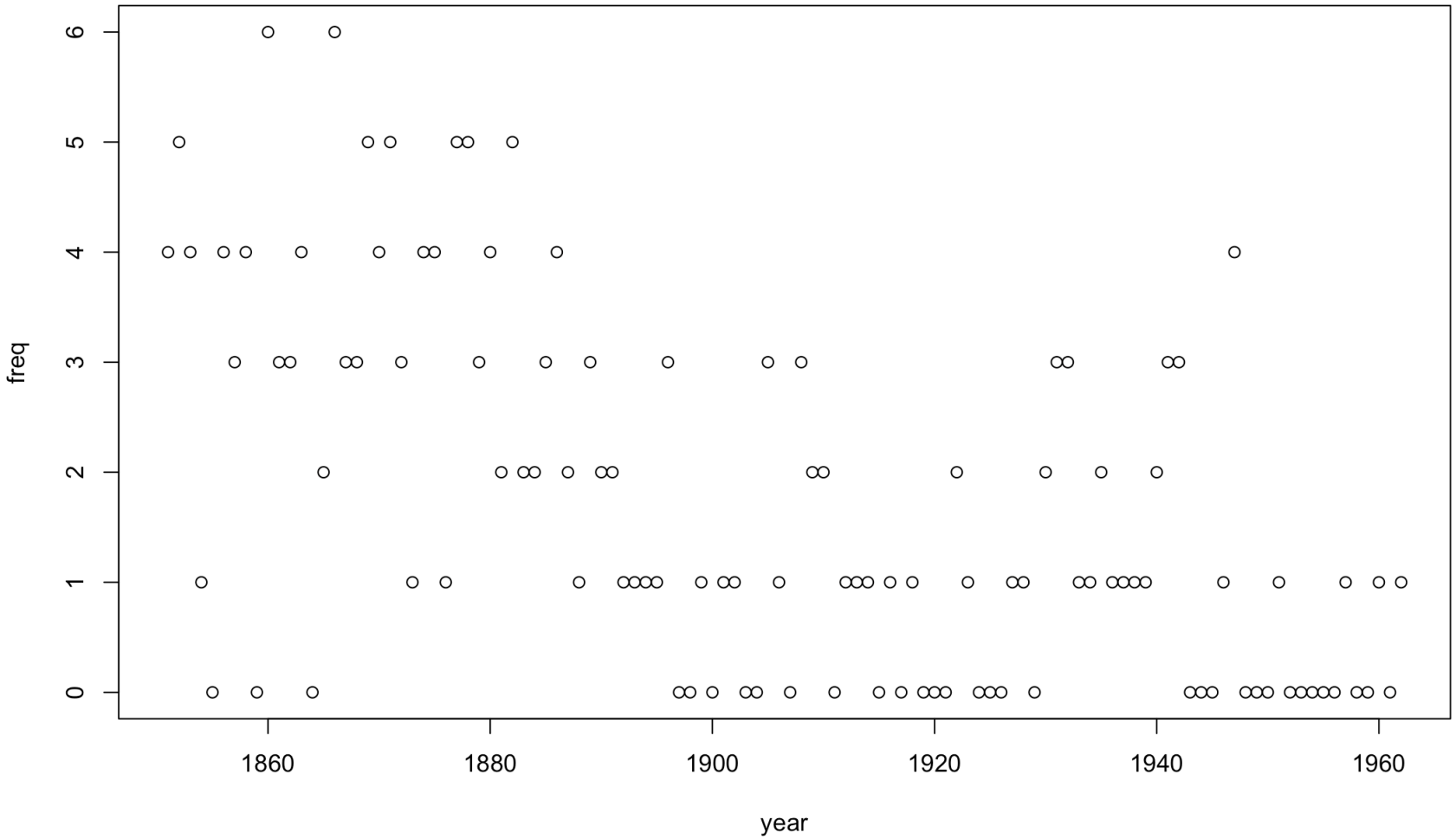
The Gibbs sampler: example

Poisson change point model

Data from Jarret (1979), A note on the intervals between coal mining disasters. *Biometrika* 66, pp. 191–193.

The data gives the dates of explosions killing 10 or more miners, from 1851 to 1962. Let Y_i be the number of such disasters in year i

```
> library(boot)
> data(coal)
> when <- floor(coal)
> year <- 1851:1962
> freq <- sapply(year, function(x, y) sum(y==x), y=when)
> n <- length(freq)
> plot(year, freq)
```


freq
0
1
2
3
4
5
6
1860
1880
1900
1920
1940
1960
year

Was there a change in the rate of disasters? To (help) answer this question we use the following model:

$$\begin{aligned} Y_i &\sim \text{pois}(\lambda_1), \text{ for } i = 1, \ldots, m \\ Y_i &\sim \text{pois}(\lambda_2), \text{ for } i = m+1, \ldots, n. \end{aligned}$$

m year index of change point.

We can impose the following priors on λ_1, λ_2, m:

$$\begin{aligned} \lambda_1 &\sim \Gamma(\alpha_1, \beta_1) \\ \lambda_2 &\sim \Gamma(\alpha_2, \beta_2) \\ m &\sim U\{1, \ldots, n\}. \end{aligned}$$

known

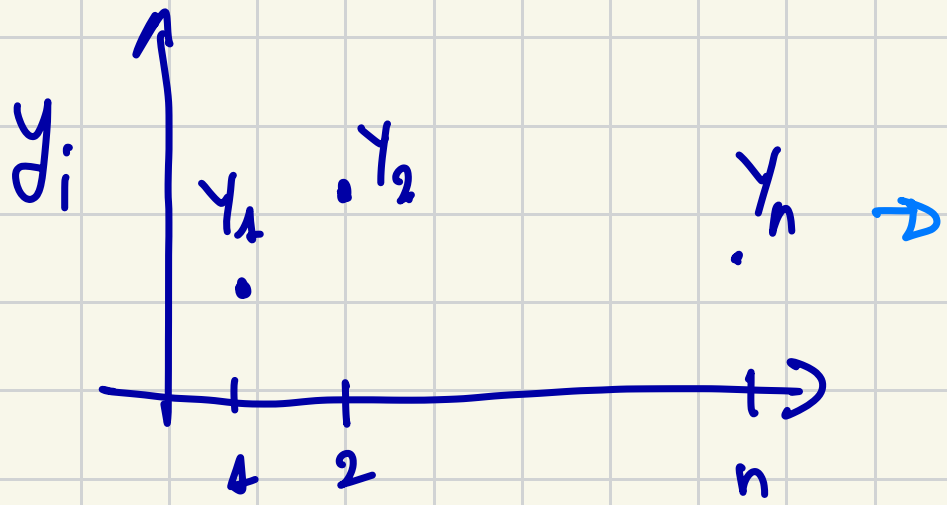
$\lambda_1 = \lambda_2$? If $\lambda_1 \neq \lambda_2$ which is the year m?

Then, the joint density is given by

$$\begin{aligned} & p(\mathrm{y}, \lambda_1, \lambda_2, m) \\ & \propto \prod_{i=1}^{m} \frac{e^{-\lambda_1} \lambda_1^{y_i}}{y_i!} \prod_{j=m+1}^{n} \frac{e^{-\lambda_2} \lambda_2^{y_j}}{y_j!} \lambda_1^{\alpha_1 - 1} e^{-\beta_1 \lambda_1} \lambda_2^{\alpha_2 - 1} e^{-\beta_2 \lambda_2}. \end{aligned}$$

y_i , $i = 1, \ldots, 112$

$1 \equiv 1851$ $\quad$ $112 \equiv 1962$



Model:

$y_i \sim \text{Poisson}(\lambda_1)$ $\quad i = 1 \ldots m$

$y_i \sim \text{Poisson}(\lambda_2)$ $\quad i = m+1 \ldots n$

☒ Are $\lambda_1 = \lambda_2$? If not, what is m ?

Inference will be based on the posterior $\rightarrow P(\lambda_1, \lambda_2, m \mid y)$

$y = y_1 \ldots y_{112}$

Let's use the bibbs sampler:

$$P(\lambda_1 \mid \lambda_2, m, y) \propto P(\lambda_1, \lambda_2, m, y)$$

$$P(\lambda_2 \mid \lambda_1, m, y) \propto P(\lambda_2, \lambda_1, m, y)$$

$$P(m \mid \lambda_1, \lambda_2, y) \propto P(m, \lambda_1, \lambda_2, y)$$

↑ focus

$$P(\lambda_1, \lambda_2, m, y) \propto P(y \mid \lambda_1, \lambda_2, m) \times P(\lambda_1, \lambda_2, m)$$

$$= \prod_{i=1}^{m} p(y_i \mid \lambda_1) \cdot \prod_{i=m+1}^{n} p(y_i \mid \lambda_2) \cdot P(\lambda_1) \cdot P(\lambda_2) P(m)$$

$$P(\lambda_1, \lambda_2, m, y) \propto P(y \mid \lambda_1, \lambda_2, m) \times P(\lambda_1, \lambda_2, m)$$

$$= \prod_{i=1}^{m} p(y_i \mid \lambda_1) \cdot \prod_{i=m+1}^{n} P(y_i \mid \lambda_2) \cdot P(\lambda_1) \cdot P(\lambda_2) P(m)$$

$$\propto \prod_{i}^{m} \frac{e^{-\lambda_1} \lambda_1^{y_i}}{y_i!} \cdot \prod_{i=m+1}^{n} \frac{e^{-\lambda_2} \lambda_2^{y_i}}{y_i!} \; \lambda_1^{\alpha_1 - 1} e^{-\beta_1 \lambda_1} \; \lambda_2^{\alpha_2 - 1} e^{-\beta_2 \lambda_2} \cdot \frac{1}{n}$$

we can ignore this

instead $y_i! = \Gamma(y_i + 1)$

↳ we can ignore this

$$= e^{-m\lambda_1} \lambda_1^{\sum_{i=1}^{m} y_i} e^{-(n-m)\lambda_2} \lambda_2^{\sum_{i=m+1}^{n} y_i}$$

$$x \sim \Gamma(\alpha, \beta)$$

$$f(x) = \frac{\beta^{\alpha}}{\Gamma(\alpha)} x^{\alpha - 1} e^{-\beta x}$$

→ Gamma density function.

- $p(\lambda_1 \mid \lambda_2, m, y) \propto e^{-m\lambda_1} \lambda_1^{\sum_{i=1}^{m} y_i} \lambda_1^{\alpha_1 - 1} e^{-\beta_1 \lambda_1}$

$= e^{-(m+b_1)\lambda_1} \lambda_1^{\sum_{i=1}^{m} y_i + \alpha_1 - 1} \rightarrow$

$\sim \Gamma\left(\sum_{i=1}^{m} y_i + \alpha_1, m + \beta_1\right)$

- $p(\lambda_2 \mid \lambda_1, m, y) \propto \dots e^{-(n-m+\beta_2)\lambda_2} \lambda_2^{\sum_{i=m+1}^{n} y_i + \alpha_2 - 1}$

$\sim \Gamma\left(\alpha_2 + \sum_{i=m+1}^{n} y_i, \beta_2 + n - m\right)$

▶ $P(m \mid \lambda_1, \lambda_2, y) \propto e^{-m\lambda_1} \lambda_1^{\sum_{i=1}^{m} y_i} \cdot e^{-n\lambda_2 + m\lambda_2} \cdot \lambda_2^{\sum_{i=1}^{n} y_i - \sum_{i=1}^{m} y_i}$

$\propto e^{(\lambda_2 - \lambda_1) m} \left(\frac{\lambda_1}{\lambda_2}\right)^{\sum_{i=1}^{m} y_i}$ (*) un-normalize weights

$= w_m$

$\sum_{i=1}^{m} y_i$ → cumsum(freq) in R

$m = 1 \ldots n$ → categorical distribution over m with weights proportional to (*)

$m = \begin{cases} 1 \\ | \\ n \end{cases}$ $p(m=1 \mid \lambda_1, \lambda_2, y)$ $\vdots$ $p(m=n \mid \lambda_1, \lambda_2, y)$

→ Tricle!

Normalizes w_i / $\sum_{i=1}^{n} w_i = 1$

▶ Therefore in the code we use `M <- sample(1:n, 1, prob=(w_1,..w_n))`

Trick

$$p(m=1 \mid \lambda_1, \lambda_2, y) \propto p(m=1, \lambda_1, \lambda_2, y) \propto e^{(\lambda_2-\lambda_1)m} \left(\frac{\lambda_1}{\lambda_2}\right)^{\sum_{i=1}^{m} y_i} \overset{(*)}{=} 19$$

$$p(m=n \mid \lambda_1, \lambda_2, y) \propto p(m=n, \lambda_1, \lambda_2, y) = 21$$

$$\Rightarrow p(m=1 \mid \lambda_1, \lambda_2, y) = \frac{19}{19 + \cdots + 21} \quad \rightarrow \text{Normalizing constant!}$$

$w_1 + \cdots + w_n$

$$p(m=n \mid \lambda_1, \lambda_2, y) = \frac{21}{19 + \cdots + 21}$$

$w_1 + \cdots + w_n$

Collecting like terms we get the posterior

$$\begin{aligned} p(\lambda_1, \lambda_2, m|\mathrm{y}) &\propto e^{-(m+\beta_1)\lambda_1} \lambda_1^{\sum_{i=1}^{m} y_i + \alpha_1 - 1} \\ &\quad \times e^{-(n-m+\beta_2)\lambda_2} \lambda_2^{\sum_{j=m+1}^{n} y_j + \alpha_2 - 1} \end{aligned}$$

and thus the conditioned distributions are

$$\begin{aligned} p(\lambda_1|\lambda_2, m, \mathrm{y}) &\sim \Gamma(\alpha_1 + \sum_{i=1}^{m} y_i, \beta_1 + m) \\ p(\lambda_2|\lambda_1, m, \mathrm{y}) &\sim \Gamma(\alpha_2 + \sum_{j=m+1}^{n} y_j, \beta_2 + n - m) \\ p(m|\lambda_1, \lambda_2, \mathrm{y}) &\propto \lambda_1^{\sum_{i=1}^{m} y_i} \lambda_2^{\sum_{j=m+1}^{n} y_j} e^{(\lambda_2 - \lambda_1)m} \\ &\propto \left(\frac{\lambda_1}{\lambda_2}\right)^{\sum_{i=1}^{m} y_i} e^{(\lambda_2 - \lambda_1)m}. \end{aligned}$$

Note that $p(m|\lambda_1, \lambda_2, \mathrm{y})$ is not a known distribution, but it is finite so we can easily simulate samples from this distribution.

We impliment a Gibbs sampler in R for this model.

See `Gibbs_example_coal.pdf`

Learning goals

Metropolis-Hastings Algorithm:

- Understand key ideas for the algorithm
- Be able to derive and implement the algorithm
- (not examinable; challenging) Obtain intuitions on the theory to support why the algorithm works

Gibbs sampler:

- Understand it is a special case of the Metropolis-Hastings Algorithm
- Understand its potential limitation compared to the Metropolis-Hastings Algorithm